Architectural Design of the Developed System

RBIM | Municipality of Tomas Oppus

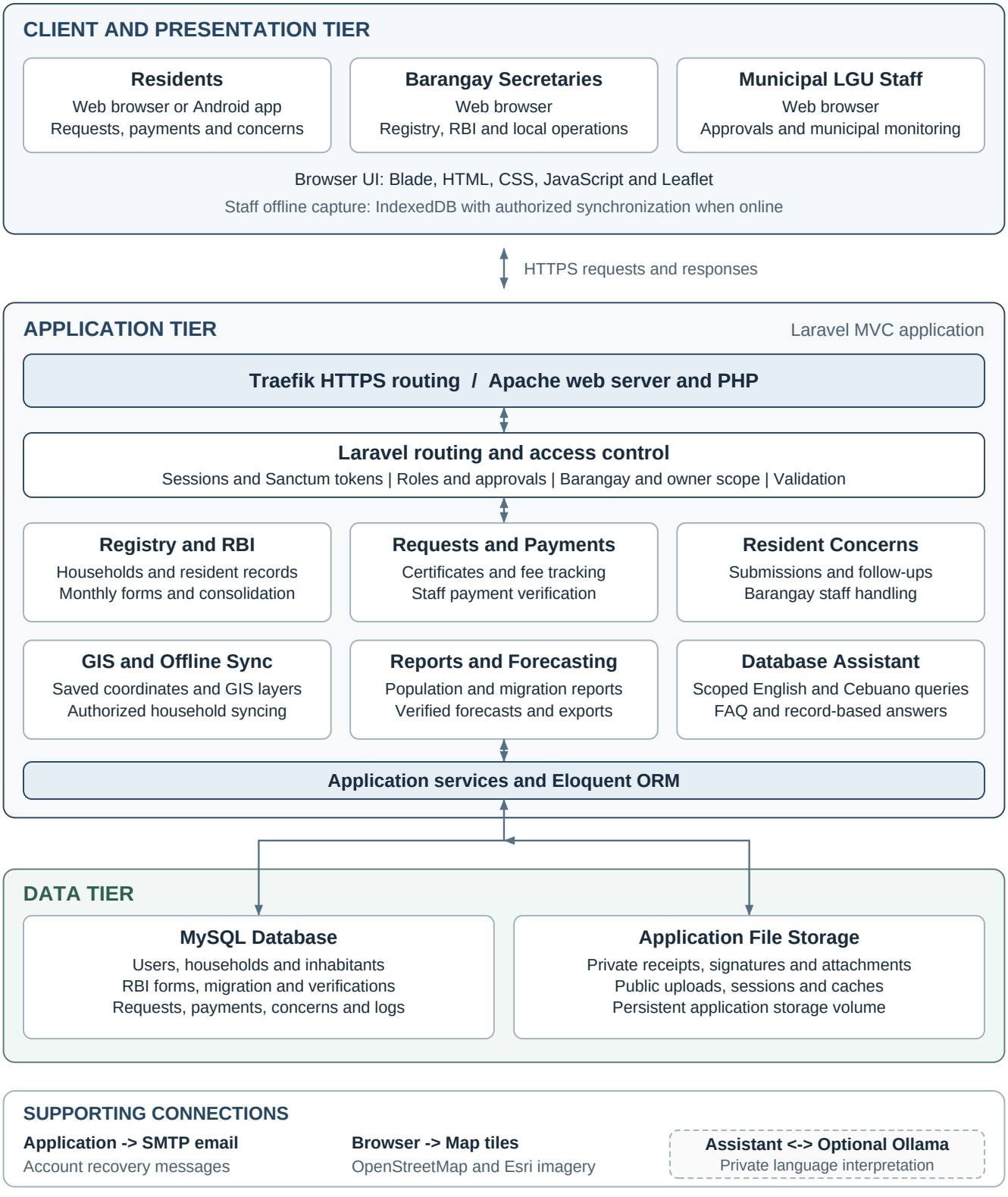

Figure 1. Architectural design of the developed RBIM system.

RBI = Registry of Barangay Inhabitants. Dashed box = optional service.

Architecture Description

Architecture overview

The developed RBIM system follows a three-tier architecture comprising the presentation tier, the application tier, and the data tier. Its Laravel application uses the Model-View-Controller pattern to separate screen rendering, request handling, and record access. Residents, barangay secretaries, and municipal LGU staff use the same central application with different permissions. The architecture supports resident services and barangay record maintenance while providing authorized municipal monitoring.

Presentation and access

The browser interface uses Blade templates, HTML, CSS, and JavaScript. Residents submit document requests, provide payment evidence, and follow up on concerns. Barangay secretaries maintain their assigned barangay records and process service transactions. Municipal LGU staff manage authorized approvals and view municipal reports; access to sensitive records remains subject to module-specific permissions. A native Android client provides resident workflows through the Laravel HTTPS API using Sanctum tokens. Browser sessions use session authentication.

Application processing

Routes and middleware check authentication, account approval, role, and barangay or record ownership before protected operations. Controllers validate submitted information and call application services. The main modules cover household and resident records, monthly RBI forms and consolidation, document requests and payment review, concern management, GIS mapping, offline synchronization, population reporting, migration forecasting, and the English and Cebuano database assistant. Eloquent models handle relational data access. Authorized download routes control access to private files.

Forecasting and supporting services

The forecasting engine runs in PHP and uses consecutive, staff-verified monthly arrival and departure counts. It compares candidate time-series methods, evaluates held-out predictions using mean absolute error, and calculates net migration as predicted arrivals minus predicted departures. Forecasts remain unavailable when verified history is insufficient. Optional Ollama support interprets assistant questions on a private connection; it does not run the numerical forecast. SMTP supports account recovery email, while the browser retrieves OpenStreetMap and Esri map tiles. GCash receipts are evidence for staff review; uploading a receipt does not execute a payment.

Data persistence and offline operation

MySQL stores accounts, households, inhabitants, RBI submissions, migration events and verification periods, document transactions, payment records, concerns, and activity history. Application storage holds private attachments, signatures, receipts, approved public files, sessions, and caches. The browser household capture workflow stores queued records in IndexedDB while offline and synchronizes them through an authenticated staff endpoint when connectivity returns. Server validation remains authoritative. The Android resident application requires connectivity and does not use this offline submission workflow.

Deployment design

The repository defines a VPS deployment managed through Dokploy. Traefik routes HTTPS traffic to an Apache and PHP application container, which connects to a separate MySQL container on a Docker network. Persistent volumes retain database files and application storage across container replacement. GitHub supplies application source for a reviewed build and redeployment. Schema migrations update the hosted database separately unless startup migrations are explicitly enabled. This deployment design describes the supplied configuration; the active online release and server settings require separate verification.